DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR

HOSTEL ACCOMMODATION AND STUDENT SERVICES MANAGEMENT SYSTEM

Program: B.Tech Computer Science Engineering (AIML)

Academic Year: 2026–27

Submitted by

Name: Shaik Hussain

Roll Number: 25WU0102100

University: Woxsen University, Hyderabad

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course.

This project provided an opportunity to connect database theory with application development. During the implementation, concepts such as relational schema design, entity-relationship modelling, normalization, primary and foreign keys, constraints, transactions, CRUD operations, database connectivity and reporting were applied to a hostel accommodation and student-services scenario.

I am grateful to the course faculty for the academic guidance and feedback provided throughout the project. I also acknowledge the technical documentation and learning resources that supported the implementation of MySQL, Python, Flask and database connectivity.

UNDERTAKING

I hereby declare that the project entitled “Design and Implementation of a Database Management System for Hostel Accommodation and Student Services Management System”, submitted as part of the Project-Based Learning activity for the Database Management Systems course, represents the project work submitted for academic evaluation.

The report has been prepared using the database schema, application source code, migration script, README documentation and testing files associated with the submitted project.

I undertake that the database structures, application workflow, SQL operations, constraints and implementation details presented in this report correspond to the submitted project. External technical concepts and documentation used during development have been acknowledged in the references.

Signature:

Name: Shaik Hussain
Roll Number: 25WU0102100
Date: 14 October 2026

ABSTRACT

The Hostel Accommodation and Student Services Management System is a database-driven web application developed to organize and manage information generated during hostel operations. Hostel administration involves interconnected activities such as student registration, hostel block and room management, bed allocation, transfers, fee schedules and payments, visitor records, complaints, maintenance requests and vacating records. Managing these activities separately can result in duplicate information, inconsistent updates and difficulty in tracking historical records.

The main objective of the project is to design and implement a centralized relational database and connect it with a web-based application for efficient hostel administration. The system uses MySQL as the database management system, Python with Flask for backend development, and HTML, CSS and JavaScript for the frontend.

The database is designed using an Entity-Relationship model and a normalized relational schema with primary keys, foreign keys, unique constraints, check constraints, indexes, generated keys, views and validation triggers. The application provides modules for dashboard monitoring, students, hostel blocks, rooms, beds, allocations, transfers, fees and payments, visitors, complaints, maintenance, vacating and reports.

The completed system demonstrates how DBMS concepts can be integrated with application development to provide structured data storage, controlled workflows, validation, transaction handling, CRUD operations and database-backed reporting for a realistic hostel-management environment.

Keywords: Hostel Management System, Relational Database, MySQL, Flask, Normalization, SQL, CRUD

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	4
2	Chapter 1 – Problem Statement, Solution Strategy and ER Diagram	9
2.1	Problem Statement and Objectives	9
2.2	Scope and Requirements	11
2.3	Solution Strategy	12
2.4	Entity-Relationship Diagram	14
3	Chapter 2 – Logical Database Design	16
3.1	Relational Schema	16
3.2	Keys and Constraints	18
3.3	Functional Dependencies	20
3.4	Normalization	21
3.5	Data Dictionary	23
3.6	Implementation of Logical Design: DDL and Sample Data	26
4	Chapter 3 – Application Development	28
4.1	Application Overview	28
4.2	Development Environment and Database Connectivity	29
4.3	Application Modules and Database Mapping	30
4.4	CRUD Operations	32
4.5	Business Rules and Validation	33
4.6	Reports and Database Operations	35
4.7	Application Screens	36
4.8	Testing and Results	42
5	Chapter 4 – Conclusion	44
5.1	Limitations and Future Enhancements	45
6	Chapter 5 – References	46
7	Chapter 6 – Appendices	47

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	11
Table 2	Proposed Solution Strategy	12
Table 3	Functional Requirements	13
Table 4	Entity and Relationship Summary	15
Table 5	Relational Schema	16
Table 6	Keys and Integrity Constraints	18
Table 7	Normalization Status up to 3NF	21
Table 8	Data Dictionary	23
Table 9	Database Implementation Environment	26
Table 10	Application Development Environment	29
Table 11	Application Module-to-Database Mapping	30
Table 12	CRUD Operations	32
Table 13	Business Rules, Validation and Error Handling	33
Table 14	Reports and Database Operations	35
Table 15	Test Cases	42
Table 16	Limitations and Future Enhancements	45

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Hostel Management System	15
Figure 2	Hostel Management System Dashboard	36
Figure 3	Students Management Screen	37
Figure 4	Hostel Blocks and Rooms Screen	38
Figure 5	Beds and Allocation Screen	39
Figure 6	Fees and Payments Screen	40
Figure 7	Visitors, Complaints and Maintenance Screens	41
Figure 8	Reports Screen	42

LIST OF ACRONYMS

Acronym	Expansion
DBMS	Database Management System
ERD	Entity-Relationship Diagram
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
REST	Representational State Transfer
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
UI	User Interface
PK	Primary Key
FK	Foreign Key
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
DDL	Data Definition Language
DML	Data Manipulation Language

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Hostel administration involves several interconnected activities such as student registration, hostel block and room management, bed allocation, transfers, fee collection, visitor monitoring, complaint handling, maintenance and vacating. When these activities are maintained separately, it becomes difficult to maintain relationships between records, avoid duplicate information and ensure that changes are applied consistently.

A relational database provides a structured approach for storing related information using tables, keys, constraints and relationships. MySQL provides facilities for defining relational schemas, enforcing integrity constraints and executing SQL-based operations.

The proposed Hostel Management System addresses this problem by maintaining hostel and student information in a centralized database. The Flask application connects the database with a web-based interface so that hostel operations can be performed through controlled application workflows.

The system also manages operational state changes. Students can be allocated to beds, transfers can be recorded, payments can be entered, visitors can be checked in and out, complaints can be updated, maintenance requests can be processed and vacating records can be maintained.

1.2 Objectives

1. To design a normalized relational database for hostel accommodation and student services.
2. To implement primary keys, foreign keys, unique constraints, check constraints, defaults and indexes.
3. To develop a Python-based backend application for interacting with the MySQL database.
4. To use Flask for backend routing and REST API services.
5. To implement modules for students, hostel blocks, rooms, beds, allocations, transfers, fees, payments, visitors, complaints, maintenance, vacating and reports.

6. To implement CRUD operations for major application entities.
7. To enforce important business rules and validation conditions.
8. To generate database-backed operational reports.
9. To demonstrate transaction handling and rollback for critical workflows.
10. To demonstrate practical integration between database design and web application development.

1.3 Scope and Assumptions

Area	Scope / Assumption
Students	Student identity, registration, contact and academic information is maintained.
Hostel Blocks	Blocks, gender policy, address and warden information are maintained.
Rooms	Room number, floor, capacity, type and active status are maintained.
Beds	Beds are associated with rooms and can be allocated to students.
Allocations	Student-to-bed allocation history and active status are maintained.
Transfers	Transfers reference source and destination allocations with reasons and approval.
Fees	Fee schedules and student payments are maintained with due dates and payment methods.
Visitors	Visitor details, purpose, check-in and check-out are recorded.
Complaints	Student complaints are categorized, prioritized and tracked to resolution.
Maintenance	Room maintenance requests are recorded with status, priority and assignment.
Vacating	Vacating records are associated with student allocations.
Reports	Occupancy, vacancies, fee dues, allocation history, visitors, complaints and maintenance reports are generated.
Authentication	Authentication/authorization workflow is outside the current academic prototype scope.

The current implementation is intended as an academic project prototype rather than a complete production hostel platform.

1.4 Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users	Hostel administrators, wardens and staff
Database	MySQL 8.0
Backend	Python with Flask
Frontend	HTML, CSS and JavaScript
Database Connectivity	MySQL Connector/Python
Major Modules	Dashboard, students, blocks, rooms, beds, allocations, transfers, fees/payments, visitors, complaints, maintenance, vacating and reports
Data Integrity	Primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, defaults, indexes and validation triggers
Reporting	SQL queries and three reporting views
Validation	Application-level and database-level validation

Main Workflow: Student → Hostel Block/Room → Bed Allocation → Services/Fees → Transfer/Vacating → Reports

11. Student information is registered.
12. Hostel blocks, rooms and beds are maintained.
13. An available bed is selected for an eligible student.
14. An allocation is created and active allocation rules are enforced.
15. Fees and payments are recorded for the student.
16. Visitors, complaints and maintenance requests can be recorded.
17. Transfers can move a student between beds through allocation history.
18. Vacating records close an allocation and preserve historical information.
19. Dashboards and reports display current and historical information.

1.5 Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	The system shall maintain student information.	High
FR-02	The system shall maintain hostel blocks and wardens.	High
FR-03	The system shall maintain rooms, capacity and room type.	High
FR-04	The system shall maintain beds and availability.	High
FR-05	The system shall create and manage student bed allocations.	High
FR-06	The system shall record and manage transfers.	High
FR-07	The system shall manage fee schedules and payments.	High
FR-08	The system shall record visitor entries and exits.	Medium
FR-09	The system shall manage complaints and maintenance requests.	High
FR-10	The system shall maintain vacating records.	High
FR-11	The system shall prevent invalid allocations, payments and dates.	High
FR-12	The system shall provide dashboard and database-backed reports.	Medium

1.6 ENTITY-RELATIONSHIP DIAGRAM

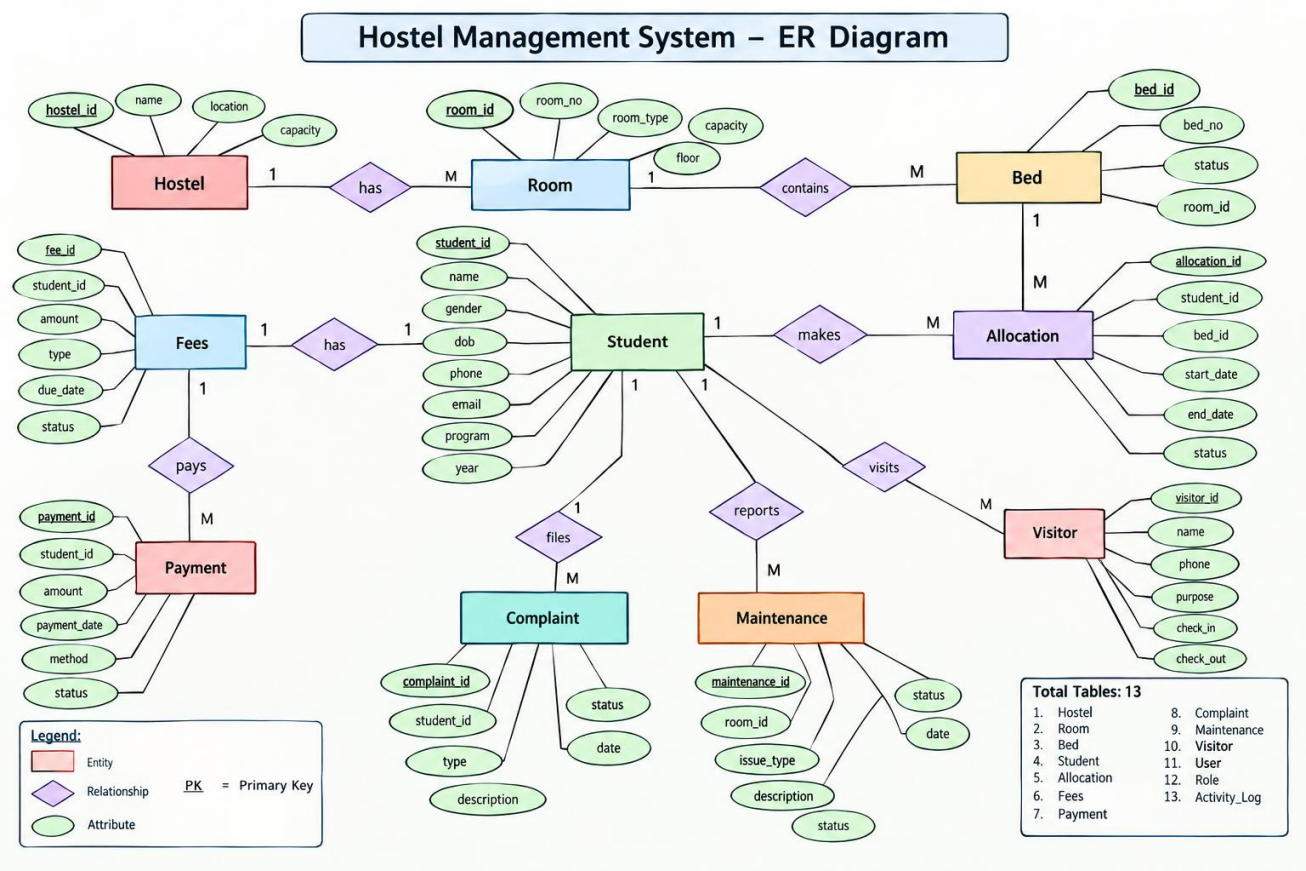


Figure 1. Entity-Relationship Diagram of the Hostel Management System

The final implemented database contains 13 tables: users, hostel_blocks, rooms, beds, students, allocations, transfers, fee_schedules, payments, visitors, complaints, maintenance_requests and vacating_records.

1.7 Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Hostel Block	Block ID, code, name, gender policy, address, warden	Stores hostel blocks.
Room	Room ID, block, number, floor, capacity, type	Stores rooms within blocks.
Bed	Bed ID, room, label, active status	Stores beds within rooms.
Student	Student ID, registration no., name, contact, department, course, year	Stores student information.
Block → Room	1:N	One block contains multiple rooms.
Room → Bed	1:N	One room contains multiple beds.
Student → Allocation	1:N history	A student can have multiple historical allocations but at most one active allocation.
Bed → Allocation	1:N history	A bed can have allocation history but at most one active allocation.
Allocation → Transfer	Source/Destination	Transfers connect old and new allocations.
Student → Payment	1:N	Payments belong to a student and fee schedule.

Student → Visitor	1:N	Visitors are associated with students.
Student → Complaint	1:N	Complaints are submitted by students.
Room → Maintenance	1:N	Maintenance requests are associated with rooms.
Allocation → Vacating	1:0/1	A vacating record closes an allocation.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model is converted into a relational schema in which entities are represented as tables and relationships are implemented using primary and foreign keys. MySQL 8.0 is used for the database implementation.

S.No.	Table Name	Primary Key	Purpose
1	users	user_id	Stores application users and roles.
2	hostel_blocks	block_id	Stores hostel block information.
3	rooms	room_id	Stores rooms within hostel blocks.
4	beds	bed_id	Stores individual beds.
5	students	student_id	Stores student information.
6	allocations	allocation_id	Stores current and historical bed allocations.
7	transfers	transfer_id	Stores allocation transfers.
8	fee_schedules	fee_schedule_id	Stores fee definitions and due dates.
9	payments	payment_id	Stores student fee payments.
10	visitors	visitor_id	Stores visitor check-in/out information.
11	complaints	complaint_id	Stores student complaints.
12	maintenance_requests	maintenance_id	Stores room maintenance requests.
13	vacating_records	vacating_id	Stores vacating information linked to allocations.

2.2 Keys and Integrity Constraints

Table	Key / Constraint	Attribute(s)	Purpose
users	PK, UNIQUE	user_id; username; email	Identifies users and prevents duplicate accounts.
hostel_blocks	PK, UNIQUE, FK	block_id; block_code; warden_user_id	Identifies blocks and links optional wardens.
rooms	PK, UNIQUE, FK, CHECK	room_id; (block_id, room_number); capacity	Prevents duplicate room numbers and invalid capacity.
beds	PK, UNIQUE, FK	bed_id; (room_id, bed_label)	Prevents duplicate bed labels within a room.

students	PK, UNIQUE, FK, CHECK	student_id; registration_no.; email; user_id; year_of_study	Maintains unique student identity and valid year.
allocations	PK, FK, UNIQUE generated, CHECK	allocation_id; student_id; bed_id; active_student_id; active_bed_id	Prevents duplicate active student/bed allocations and invalid dates.
transfers	PK, UNIQUE, FK, CHECK	transfer_id; source/destination allocation IDs	Maintains transfer history and prevents same source/destination.
fee_schedules	PK, UNIQUE, CHECK	fee_schedule_id; academic year/term/type; amount	Prevents duplicate fee definitions and invalid amounts.
payments	PK, UNIQUE, FK, CHECK	payment_id; transaction_reference; student_id; fee_schedule_id; amount	Maintains valid positive payments.
visitors	PK, FK, CHECK	visitor_id; student_id; check-in/out	Maintains visitor history and valid times.
complaints	PK, FK, CHECK	complaint_id; student_id; assigned_to; resolved_at	Maintains complaint workflow and valid resolution dates.
maintenance_requests	PK, FK, CHECK	maintenance_id; room_id; requested_by_student_id; completed_at	Maintains maintenance workflow.
vacating_records	PK, UNIQUE, FK, CHECK	vacating_id; allocation_id	Maintains one vacating record per allocation and valid dates.

The database uses primary keys for row identity, foreign keys for referential integrity, UNIQUE constraints for duplicate prevention, NOT NULL for mandatory values, CHECK constraints for domain validation, default values for automatic states, and indexes for common lookups.

2.3 Functional Dependencies

Relation	Functional Dependency
User	user_id → username, password_hash, full_name, email, role, is_active
Hostel Block	block_id → block_code, block_name, gender_policy, address, warden_user_id, is_active
Room	room_id → block_id, room_number, floor_number, capacity, room_type, is_active
Bed	bed_id → room_id, bed_label, is_active
Student	student_id → registration_no, first_name, last_name, email, phone, gender, DOB, department, course, year, emergency contact
Allocation	allocation_id → student_id, bed_id, allocation_date, expected_vacate_date, actual_vacate_date, status, created_by
Transfer	transfer_id → from_allocation_id, to_allocation_id, transfer_date, reason, approved_by
Fee Schedule	fee_schedule_id → academic_year, term, fee_type, amount, due_date, is_active
Payment	payment_id → student_id, fee_schedule_id, amount, payment_date, payment_method, transaction_reference
Visitor	visitor_id → student_id, visitor details, purpose, check_in, check_out, approved_by
Complaint	complaint_id → student_id, category, subject, description, status, priority, submitted_at, resolved_at, assigned_to

Maintenance	maintenance_id → room_id, requested_by_student_id, category, description, status, priority, requested_at, completed_at, assigned_to
Vacating	vacating_id → allocation_id, vacate_date, reason, processed_by, notes

These dependencies support the identification of candidate keys and normalization. Transfer student identity is not duplicated in the transfer relation; it is derived through the source allocation, avoiding a 3NF violation.

2.4 Normalization up to Third Normal Form

Normalization is used to reduce redundancy and avoid insertion, update and deletion anomalies.

First Normal Form — 1NF

The database satisfies 1NF because attributes contain atomic values and repeating groups are not stored inside individual fields. Student services such as visitors and complaints are stored as separate rows rather than multiple values in one student record.

Second Normal Form — 2NF

The database satisfies 2NF because non-key attributes depend on the complete primary key. Independent entities such as students, rooms, beds, fees and payments are separated into their own relations.

Third Normal Form — 3NF

The database satisfies 3NF because non-key attributes depend on the primary key and not on other non-key attributes. Transfer records reference allocation identifiers rather than repeating student identity.

Relation	1NF	2NF	3NF	Remarks
users	Satisfied	Satisfied	Satisfied	User details depend on user_id.
hostel_blocks	Satisfied	Satisfied	Satisfied	Block attributes depend on block_id.
rooms	Satisfied	Satisfied	Satisfied	Room attributes depend on room_id.
beds	Satisfied	Satisfied	Satisfied	Bed attributes depend on bed_id.
students	Satisfied	Satisfied	Satisfied	Student details depend on student_id.
allocations	Satisfied	Satisfied	Satisfied	Allocation details depend on allocation_id.
transfers	Satisfied	Satisfied	Satisfied	Transfer details depend on transfer_id; student is derived via allocation.
fee_schedules	Satisfied	Satisfied	Satisfied	Fee attributes depend on fee_schedule_id.
payments	Satisfied	Satisfied	Satisfied	Payment attributes depend on payment_id.
visitors	Satisfied	Satisfied	Satisfied	Visitor attributes depend on visitor_id.
complaints	Satisfied	Satisfied	Satisfied	Complaint attributes

				depend on complaint_id.
maintenance_requests	Satisfied	Satisfied	Satisfied	Maintenance attributes depend on maintenance_id.
vacating_records	Satisfied	Satisfied	Satisfied	Vacating attributes depend on vacating_id.

Example of Normalization

An unnormalized hostel record might combine student details, room details, bed details, payment details and complaint information in one large relation. This would duplicate student and room information.

```
Unnormalized:
HostelRecord(student_id, student_name, room_no, bed_label,
             fee_amount, payment_amount, complaint_subject)

Normalized:
Student(student_id, registration_no, name, contact, course, year)
Room(room_id, block_id, room_number, capacity)
Bed(bed_id, room_id, bed_label)
Allocation(allocation_id, student_id, bed_id, dates, status)
FeeSchedule(fee_schedule_id, fee_type, amount, due_date)
Payment(payment_id, student_id, fee_schedule_id, amount, method)
Complaint(complaint_id, student_id, subject, description, status)
```

2.5 Data Dictionary

Table	Important Attribute	Data Type	Key / Constraint	Description
users	user_id	BIGINT UNSIGNED	PK	Unique user identifier
users	username	VARCHAR(50)	UNIQUE, NOT NULL	Login/user name
hostel_blocks	block_id	BIGINT UNSIGNED	PK	Unique block identifier
hostel_blocks	gender_policy	ENUM	NOT NULL	Male/Female/Mixed policy
rooms	room_id	BIGINT UNSIGNED	PK	Unique room identifier
rooms	capacity	TINYINT UNSIGNED	CHECK	Maximum room capacity
beds	bed_id	BIGINT UNSIGNED	PK	Unique bed identifier
beds	bed_label	VARCHAR(10)	UNIQUE	Bed label within room
students	student_id	BIGINT UNSIGNED	PK	Unique student identifier
students	registration_no	VARCHAR(30)	UNIQUE, NOT NULL	University registration number
students	course	VARCHAR(100)	NOT NULL	Student course
students	year_of_study	TINYINT UNSIGNED	CHECK	Year between 1 and 8
allocations	allocation_id	BIGINT UNSIGNED	PK	Allocation identifier
allocations	status	ENUM	NOT NULL	ACTIVE/TRANSFERRED/VACATED/CANCELLED
transfers	transfer_id	BIGINT UNSIGNED	PK	Transfer identifier
fee_schedules	fee_schedule_id	BIGINT UNSIGNED	PK	Fee schedule identifier
payments	payment_id	BIGINT	PK	Payment identifier

		UNSIGNED		
visitors	visitor_id	BIGINT UNSIGNED	PK	Visitor record identifier
complaints	complaint_id	BIGINT UNSIGNED	PK	Complaint identifier
maintenance_requests	maintenance_id	BIGINT UNSIGNED	PK	Maintenance identifier
vacating_records	vacating_id	BIGINT UNSIGNED	PK	Vacating record identifier

2.6 IMPLEMENTATION OF LOGICAL DESIGN: DDL AND SAMPLE DATA

Item	Details
DBMS	MySQL 8.0
Database Name	hostel_management
Operational Tables	13
Reporting Views	3
Stored Procedures	0
Validation Triggers	8
Backend Language	Python
Application Framework	Flask
Frontend	HTML, CSS, JavaScript
Connector	mysql-connector-python

Representative DDL:

```
CREATE TABLE students (
    student_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    registration_no VARCHAR(30) NOT NULL UNIQUE,
    first_name VARCHAR(70) NOT NULL,
    last_name VARCHAR(70) NOT NULL,
    email VARCHAR(254) NOT NULL UNIQUE,
    phone VARCHAR(25) NOT NULL,
    gender ENUM('MALE','FEMALE','OTHER') NOT NULL,
    department VARCHAR(100) NOT NULL,
    course VARCHAR(100) NOT NULL,
    year_of_study TINYINT UNSIGNED NOT NULL,
    emergency_contact_name VARCHAR(120) NOT NULL,
    emergency_contact_phone VARCHAR(25) NOT NULL,
    CONSTRAINT ck_students_year
        CHECK (year_of_study BETWEEN 1 AND 8)
);
```

Sample data is provided for users, hostel blocks, rooms, beds, students, allocations, transfers, fee schedules, payments, visitors, complaints, maintenance requests and vacating records.

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The Hostel Management System is implemented as a web application that provides a user interface for hostel administration. The frontend communicates with the Python Flask backend through API endpoints. Flask manages routing and backend request handling, while Python provides application logic and validation.

The backend communicates with MySQL to create, retrieve, update and manage hostel information. The application is therefore directly connected to the logical database design rather than relying on isolated static data.

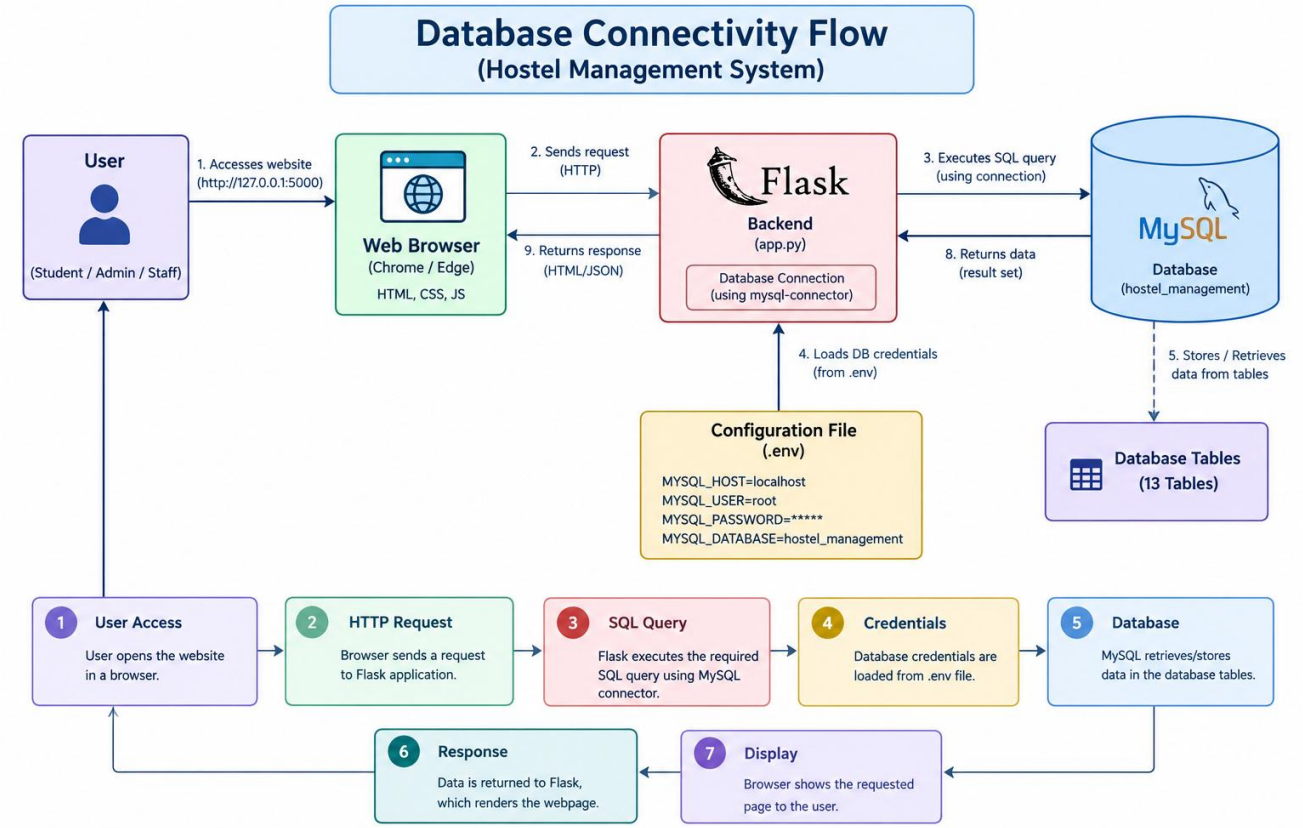
Major application modules include:

20. Dashboard
21. Students
22. Hostel Blocks
23. Rooms
24. Beds
25. Allocations
26. Transfers
27. Fees & Payments
28. Visitors
29. Complaints
30. Maintenance
31. Vacating
32. Reports

3.2 Development Environment and Database Connectivity

Item	Details
Programming Language	Python
Backend Framework	Flask
Database	MySQL 8.0
Frontend	HTML, CSS, JavaScript
Database Connector	MySQL Connector/Python
Development Environment	Antigravity / Windows
Operating Environment	Windows
Database Name	hostel_management
Repository	GitHub – HUSSAIN-134/HostelManagementSystem

Database Connectivity Flow:



Browser → Flask API → Database Layer → MySQL hostel_management

Local database credentials are supplied through a private .env file. The .env file is excluded from GitHub using .gitignore, while .env.example documents the required variable names without exposing secrets.

3.3 Application Modules and Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Operational summary	Multiple tables/views	Read/Aggregation
Students	Manage student information	students, users	CRUD
Hostel Blocks	Manage blocks and wardens	hostel_blocks, users	CRUD
Rooms	Manage rooms and capacity	rooms, hostel_blocks	CRUD
Beds	Manage beds and availability	beds, rooms, allocations	CRUD/Status
Allocations	Assign students to beds	allocations, students, beds	Create/Read
Transfers	Move students between allocations	transfers, allocations	Create/Read
Fees & Payments	Manage fee schedules and payments	fee_schedules, payments, students	Create/Read
Visitors	Record visitor entry/exit	visitors, students	Create/Read/Update
Complaints	Track complaints	complaints, students, users	CRUD/Status
Maintenance	Track room maintenance	maintenance_requests, rooms, users	CRUD/Status

Vacating	Record student vacating	vacating_records, allocations	Create/Read
Reports	Display operational analysis	3 reporting views + SQL queries	Read/Aggregation

3.4 CRUD Operations

Operation	Application Feature / Screen	Tables Affected	Evidence
Create	Student registration	students	Student entry screen
Create	Hostel block	hostel_blocks	Block entry screen
Create	Room / Bed	rooms, beds	Room/Bed screens
Create	Allocation	allocations	Allocation screen
Create	Transfer	transfers, allocations	Transfer screen
Create	Fee / Payment	fee_schedules, payments	Fees & Payments screen
Create	Visitor	visitors	Visitor screen
Create	Complaint / Maintenance	complaints, maintenance_requests	Service screens
Read	Dashboard	Multiple tables/views	Dashboard
Read	Reports	Views and SQL	Reports screen
Update	Student / Block / Room / Bed	Relevant tables	Edit forms
Update	Complaint / Maintenance	Relevant tables	Status workflow
Update	Visitor checkout	visitors	Checkout action
Delete / Controlled Removal	Applicable master records	Relevant tables	Restricted by FK/business rules

Destructive operations are intentionally constrained where deletion could compromise historical allocation, financial, complaint or maintenance records. Foreign-key restrictions help preserve relational integrity.

3.5 Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Expected Behaviour
Unique active student allocation	Generated unique key on active_student_id	A student cannot have more than one active allocation.
Unique active bed allocation	Generated unique key on active_bed_id	A bed cannot be allocated to more than one student at a time.
Room capacity	Database trigger/application validation	Allocations cannot exceed available room capacity.
Block gender policy	Validation trigger	Student gender must comply with the block's policy.
Allocation dates	CHECK + API validation	Expected vacate date must not precede allocation date.
Transfer dates	API validation	Transfer date must be valid relative to allocation history.
Payment amount	CHECK + trigger/API validation	Payment must be positive and cannot exceed the applicable fee balance.
Visitor times	CHECK	Check-out cannot precede check-in.
Complaint resolution	CHECK	Resolution date cannot precede submission date.
Maintenance workflow	API validation	Completion state requires valid workflow dates/status.

Vacating workflow	Transaction + validation	Vacating updates allocation state consistently.
Database failure	Exception handling + rollback	Failed transactional operations roll back their database changes.

SQL statements are parameterized so user input is not directly concatenated into SQL commands. This improves query safety and reduces SQL injection risk.

3.6 Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Occupancy Report	v_occupancy / aggregation	Current hostel occupancy
Vacancy Report	v_vacant_beds	Available beds
Fee Dues Report	v_fee_dues	Outstanding fee amounts
Allocation History	JOIN / ordering	Student allocation history
Visitor Report	JOIN / filtering	Visitor activity
Complaint Report	Filtering / aggregation	Open and unresolved complaints
Maintenance Report	Filtering / ordering	Maintenance workload and status

The database contains three reporting views: v_occupancy, v_vacant_beds and v_fee_dues. The SQL query catalog also demonstrates SELECT/WHERE/ORDER BY, INSERT/UPDATE/DELETE, joins, grouping/HAVING, aggregate functions, subqueries and reporting operations.

3.7 APPLICATION SCREENS

Insert final screenshots of the running application in the spaces below. Screenshots should show data entry, search/display, update/transaction processing and report/output evidence.

DASHBOARD

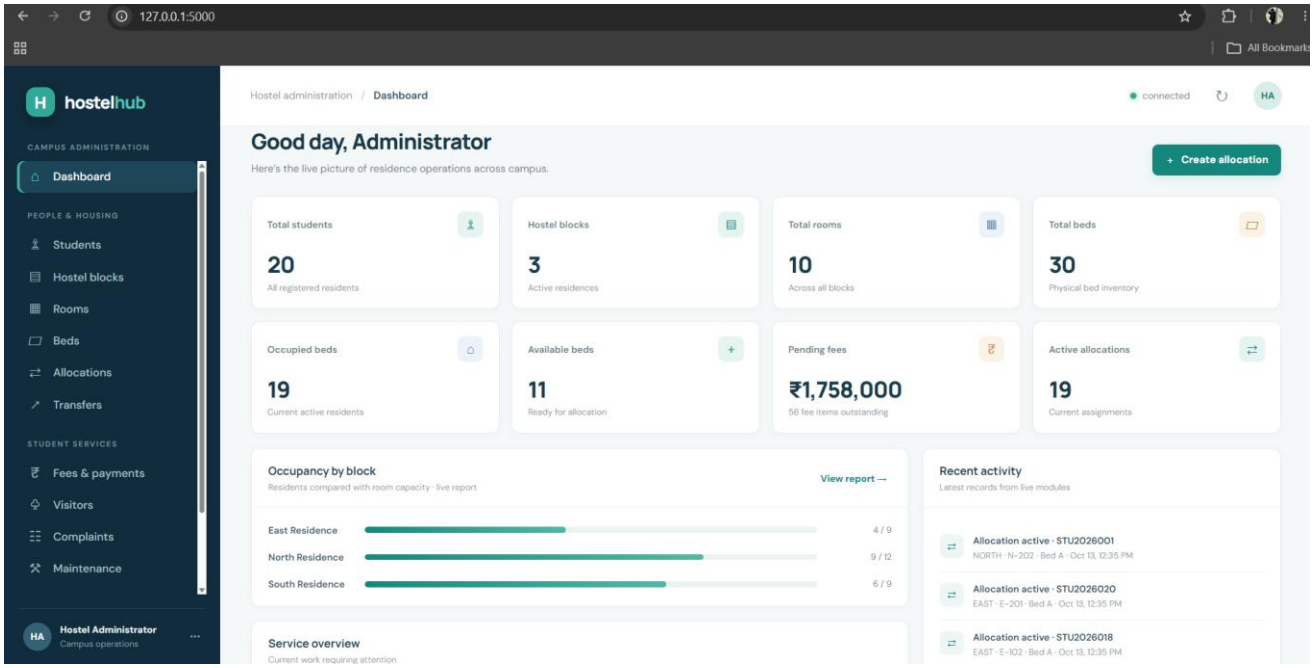


Figure 2. Hostel Management System Dashboard

Description: This screen demonstrates the dashboard functionality and its connection to the underlying MySQL database.

STUDENTS MANAGEMENT

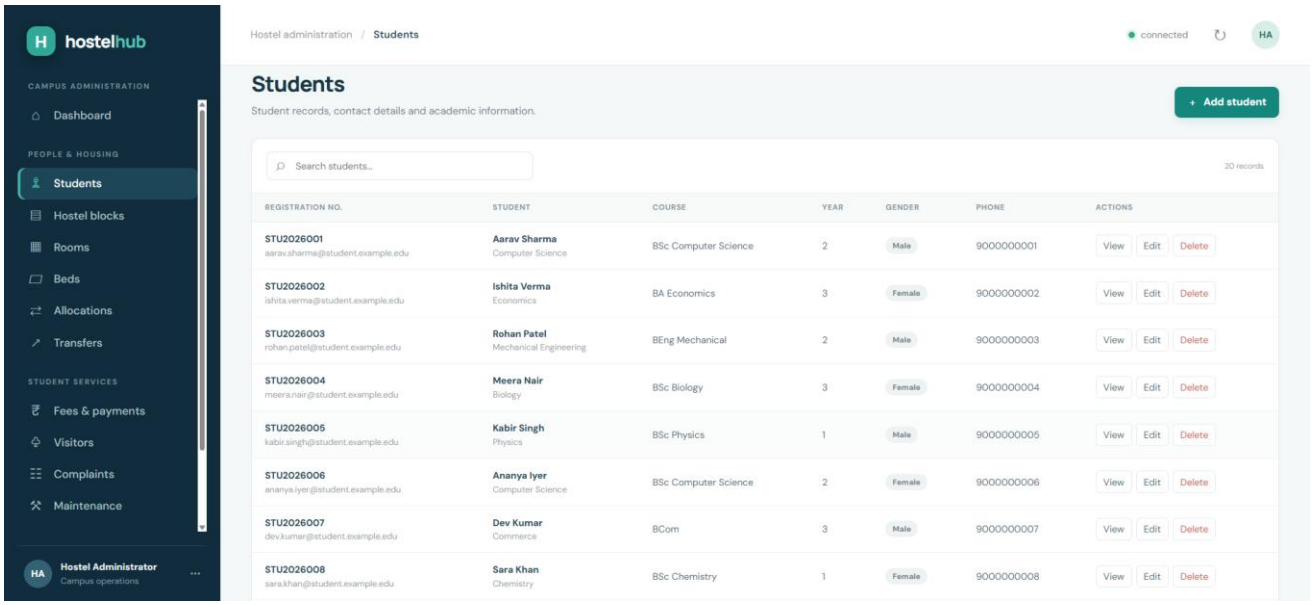


Figure 3. Students Management Screen

Description: This screen demonstrates the students management functionality and its connection to the underlying MySQL database.

HOSTEL BLOCKS AND ROOMS

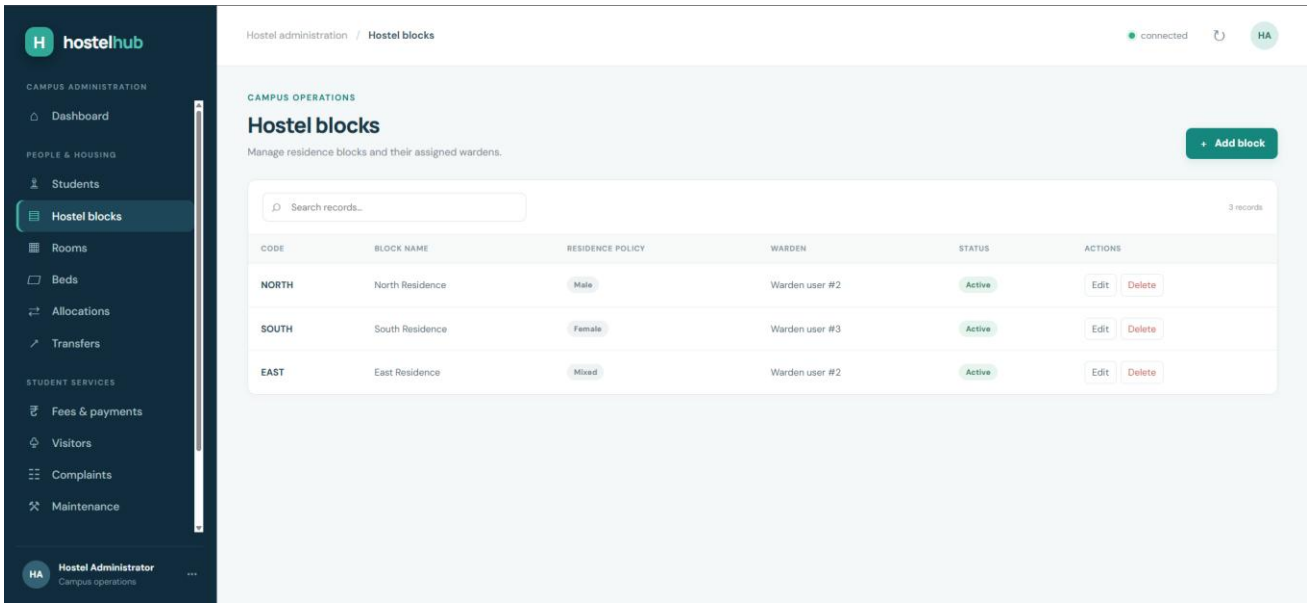


Figure 4. Hostel Blocks and Rooms Screen

Description: This screen demonstrates the hostel blocks and rooms functionality and its connection to the underlying MySQL database.

BEDS AND ALLOCATIONS

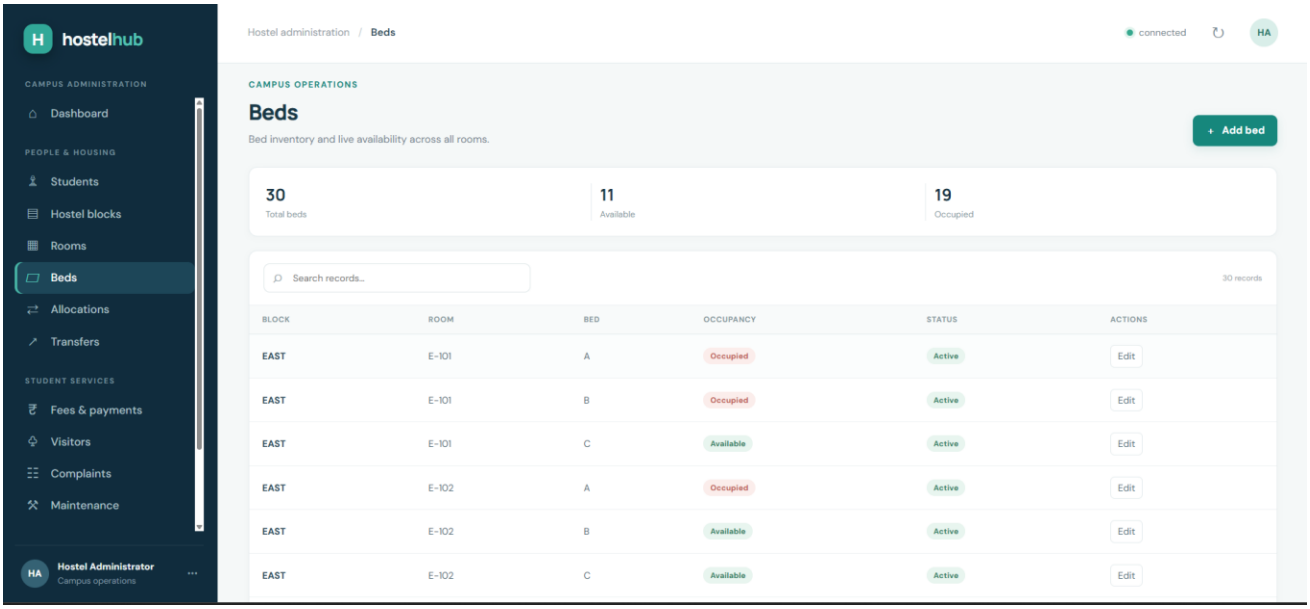


Figure 5. Beds and Allocation Screen

Description: This screen demonstrates the beds and allocations functionality and its connection to the underlying MySQL database.

FEES AND PAYMENTS

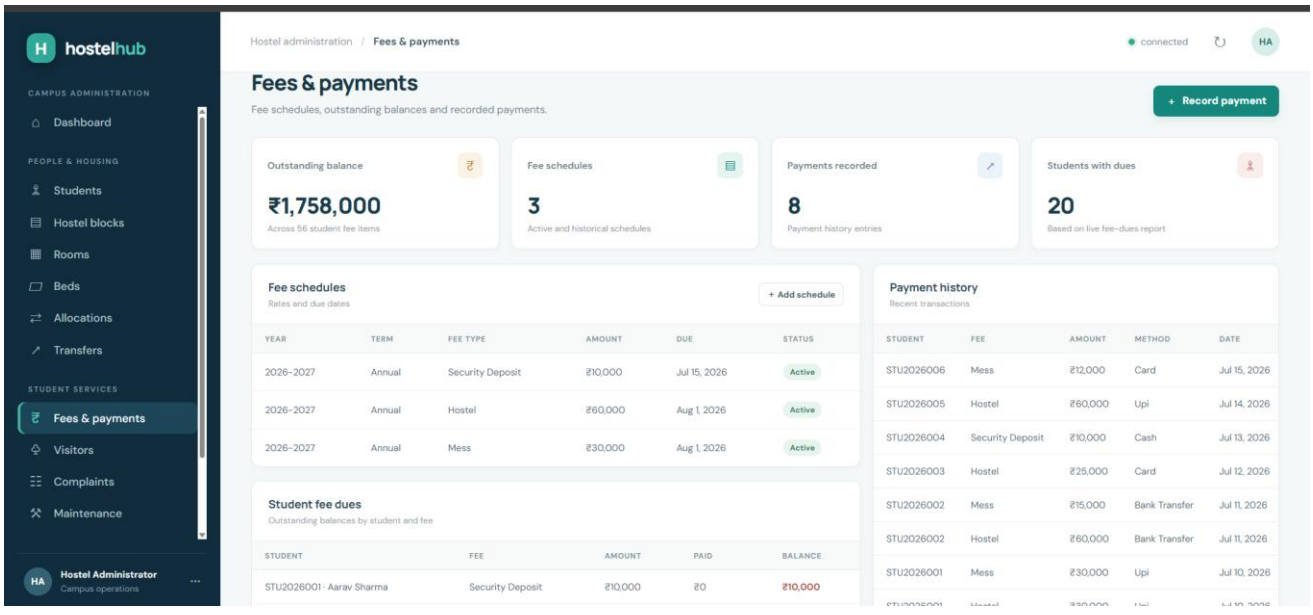


Figure 6. Fees and Payments Screen

Description: This screen demonstrates the fees and payments functionality and its connection to the underlying MySQL database.

VISITORS, COMPLAINTS AND MAINTENANCE

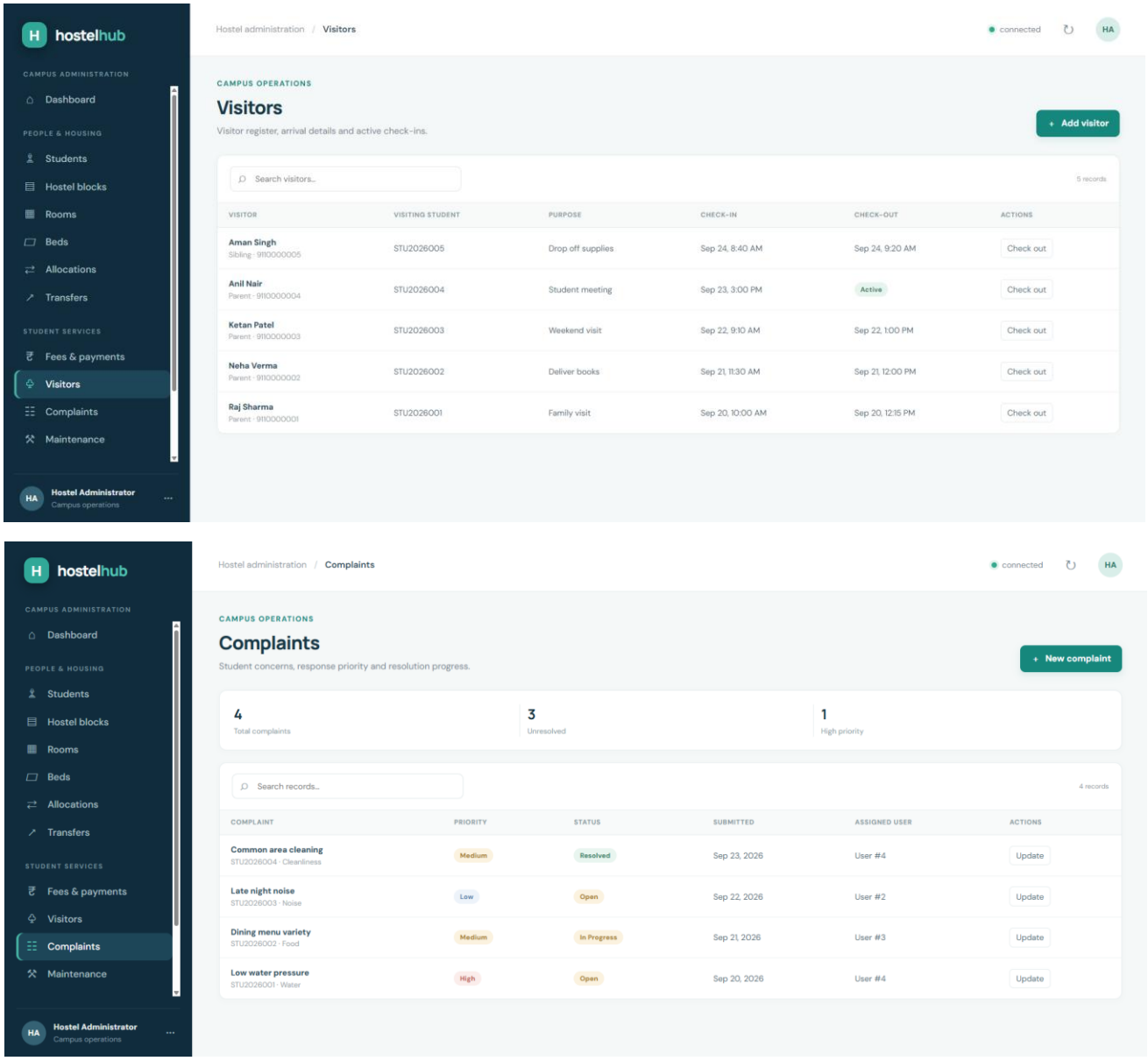


Figure 7. Student Services Screens

Description: This screen demonstrates the visitors, complaints and maintenance functionality and its connection to the underlying MySQL database.

REPORTS

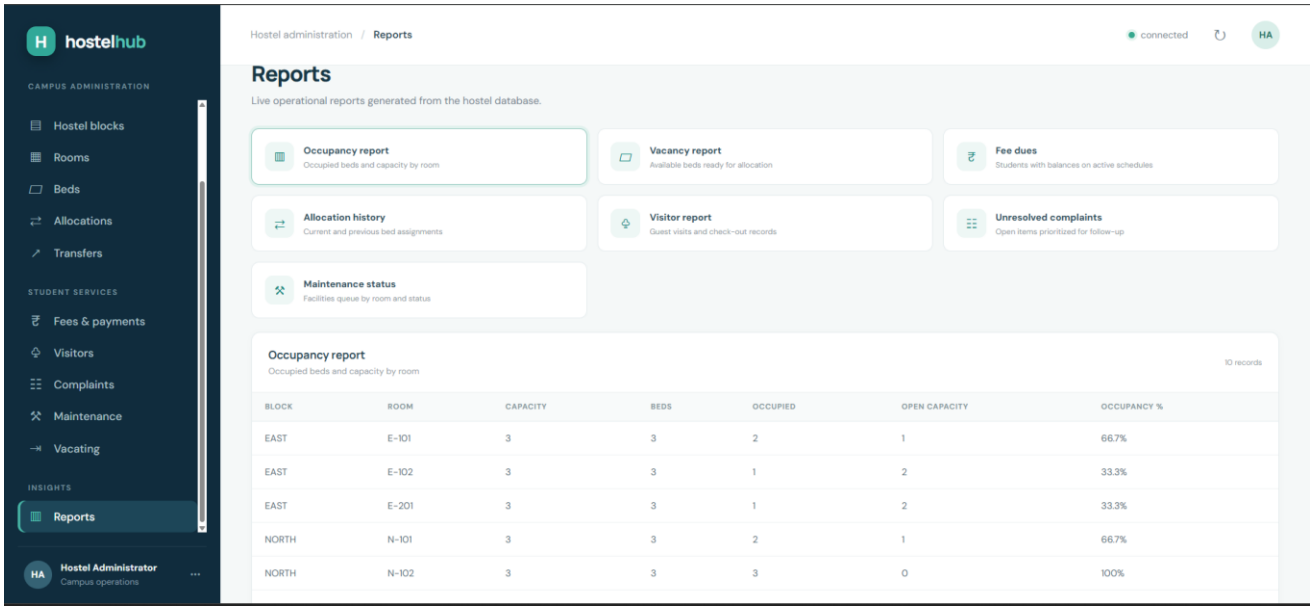


Figure 8. Reports Screen

Description: This screen demonstrates the reports functionality and its connection to the underlying MySQL database.

3.8 TESTING AND RESULTS

Testing was performed using positive and negative scenarios covering API reads, CRUD operations, invalid inputs, allocation, transfer, vacating, fees, payments, visitors, complaints and maintenance. The final project audit reported successful integration tests against the configured MySQL database.

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Database health	GET /api/health	Database connection should be reported as connected.	HTTP 200; connected	PASS
TC-02	Dashboard statistics	GET /api/stats	Statistics should be returned from MySQL.	HTTP 200; expected counts returned	PASS
TC-03	Student CRUD	Create/update/delete temporary student	Valid CRUD operations should complete.	CRUD smoke test completed; count restored	PASS
TC-04	Invalid student	Invalid year / duplicate registration	Request should be rejected.	Validation/duplicate error returned	PASS
TC-05	Allocation	Create valid allocation	Student receives an available bed.	Allocation created and active rules enforced	PASS
TC-06	Transfer	Transfer active student to another bed	Source allocation should close and destination become active.	Transfer workflow completed	PASS

TC-07	Vacating	Vacate active allocation	Allocation should be vacated consistently.	Vacating workflow completed	PASS
TC-08	Payment validation	Invalid or over-balance payment	Payment should be rejected.	Validation/constraint rejection	PASS
TC-09	Service records	Visitor/complaint/maintenance write	Valid records should be stored.	Records created successfully	PASS
TC-10	Rollback	Force failed transfer/vacating operation	Original allocation state should remain unchanged.	Rollback verified	PASS

The project audit also verified that all module and report GET endpoints returned HTTP 200, the Python files and tests compiled successfully, and the frontend calls Flask APIs rather than using hardcoded database arrays.

CHAPTER 4

CONCLUSION

The Hostel Accommodation and Student Services Management System demonstrates the development of a database-backed web application for managing interconnected hostel operations. The system converts activities such as student registration, block and room management, bed allocation, transfers, fee management, visitor monitoring, complaints, maintenance and vacating into structured relational data.

The database is implemented using MySQL 8.0 with 13 operational tables, primary keys, foreign keys, integrity constraints, three reporting views and eight validation triggers. Python is used for application logic, while Flask provides the backend application and API layer. The frontend provides interfaces for managing hostel activities and displaying database-backed information.

The project demonstrates important DBMS concepts including ER modelling, relational schema design, functional dependencies, normalization up to 3NF, SQL DDL/DML, CRUD operations, constraints, validation, reporting, database connectivity and transaction-oriented application workflows.

Overall, the project provides practical experience in designing and implementing a normalized relational database and integrating it with a web application to solve a realistic hostel-management problem.

4.1 LIMITATIONS AND FUTURE ENHANCEMENTS

Current Limitation	Future Enhancement
Authentication/authorization is not implemented	Add secure login, session management and role-based access control.
Production security is limited	Use HTTPS, secure secret management, stronger deployment configuration and audit logging.
No production-scale concurrency testing	Perform load and concurrent transaction testing.
Reporting scope is academic	Add date-range, occupancy trends, fee trends and comparative dashboards.
External services are not integrated	Future versions could integrate email/SMS notifications and payment services.
Backup and recovery are outside the prototype	Add automated backup, monitoring and recovery procedures.
Scalability is not evaluated under production load	Add performance testing, indexing analysis, caching and deployment monitoring.

CHAPTER 5

REFERENCES

[1] Oracle Corporation, “MySQL 8.0 Reference Manual,” MySQL Documentation.

[2] Python Software Foundation, “Python 3 Documentation,” Python Documentation.

[3] Pallets, “Flask Documentation,” Flask Documentation.

[4] Oracle Corporation, “MySQL Connector/Python Developer Guide.”

[5] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, relational database design and normalization concepts.

CHAPTER 6

APPENDICES

APPENDIX A — COMPLETE DATABASE SCRIPT

The submitted project contains executable SQL files for database creation, table definitions, constraints, sample data, migration and reporting queries.

A.1 Database Name

Database Name: hostel_management

A.2 Operational Tables

- users
- hostel_blocks
- rooms
- beds
- students
- allocations
- transfers
- fee_schedules
- payments
- visitors
- complaints
- maintenance_requests
- vacating_records

A.3 Database Objects

Database Object	Count
Operational Tables	13
Reporting Views	3
Stored Procedures	0
Validation Triggers	8

A.4 Primary Keys

Table	Primary Key
users	user_id
hostel_blocks	block_id
rooms	room_id
beds	bed_id
students	student_id
allocations	allocation_id
transfers	transfer_id
fee_schedules	fee_schedule_id

payments	payment_id
visitors	visitor_id
complaints	complaint_id
maintenance_requests	maintenance_id
vacating_records	vacating_id

A.5 Constraints

- Primary Key constraints uniquely identify records.
- Foreign Key constraints maintain relationships between related tables.
- UNIQUE constraints prevent duplicate values where required.
- NOT NULL constraints ensure mandatory information is supplied.
- CHECK constraints validate dates, amounts, capacities and domains.
- DEFAULT values provide automatic initial states.
- Generated unique keys enforce at most one active allocation per student and per bed.

A.6 Sample Data

The sample database contains users, students, hostel blocks, rooms, beds, allocation history, fees, payments, visitors, complaints and maintenance records for demonstration and testing.

A.7 Reporting Views

- v_occupancy — occupancy summary.
- v_vacant_beds — currently available beds.
- v_fee_dues — fee due/balance information.

A.8 Validation Triggers

- Room/bed capacity validation.
- Hostel block gender-policy validation.
- Fee payment cap validation.
- Allocation/vacating date and state validation.
- Additional database-level integrity rules implemented in the schema.

APPENDIX B — APPLICATION SOURCE CODE

Project Repository:

https://github.com/HUSSAIN-134/HostelManagementSystem

The repository contains the Flask backend, database scripts, frontend files, documentation and integration tests.

Major Project Structure

```
HostelManagementSystem/
├── app.py
├── database.py
├── requirements.txt
├── README.md
├── database/
│   ├── schema.sql
│   ├── sample_data.sql
│   ├── queries.sql
│   └── migrations/
│       └── 001_normalize_transfers.sql
├── docs/
│   ├── VIVA.md
│   └── PROJECT_AUDIT.md
├── templates/
│   └── index.html
├── static/
│   ├── css/app.css
│   └── js/app.js
└── tests/
    └── test_api.py
```

APPENDIX C — APPLICATION SCREENSHOTS

Insert final screenshots from the working localhost application in the following spaces. Use clear screenshots with captions below each figure.

C.1 Dashboard

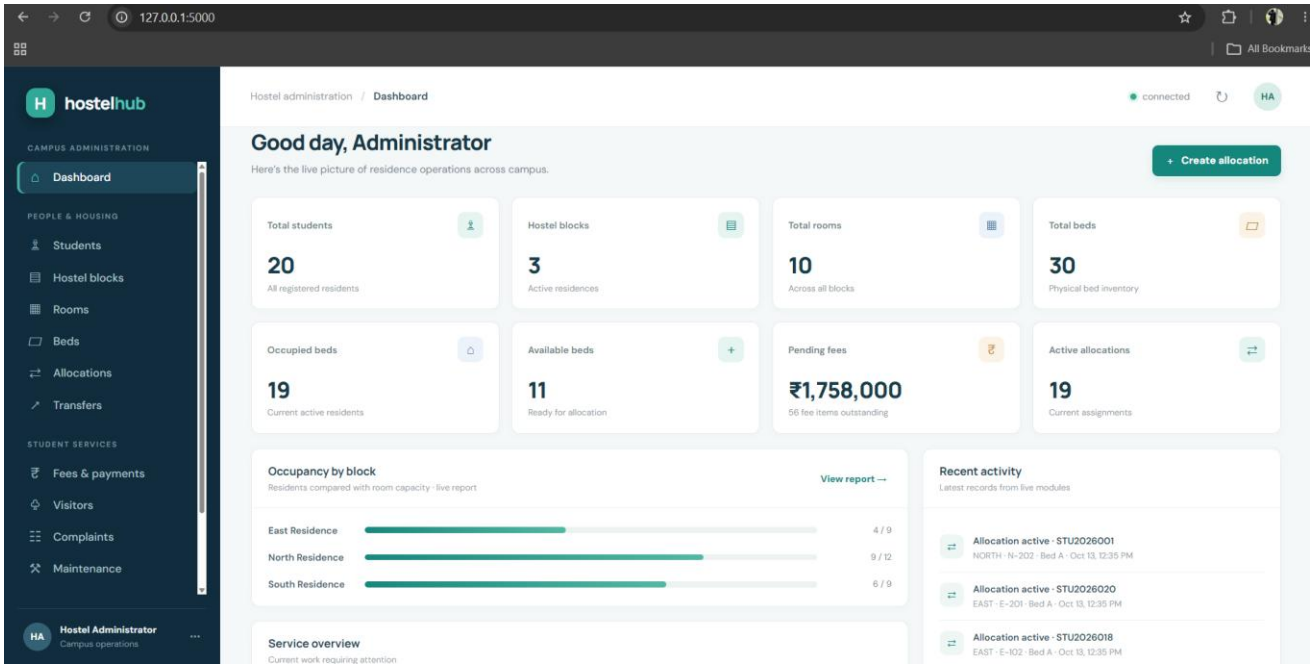


Figure C.1. C.1 Dashboard

C.2 Students Management

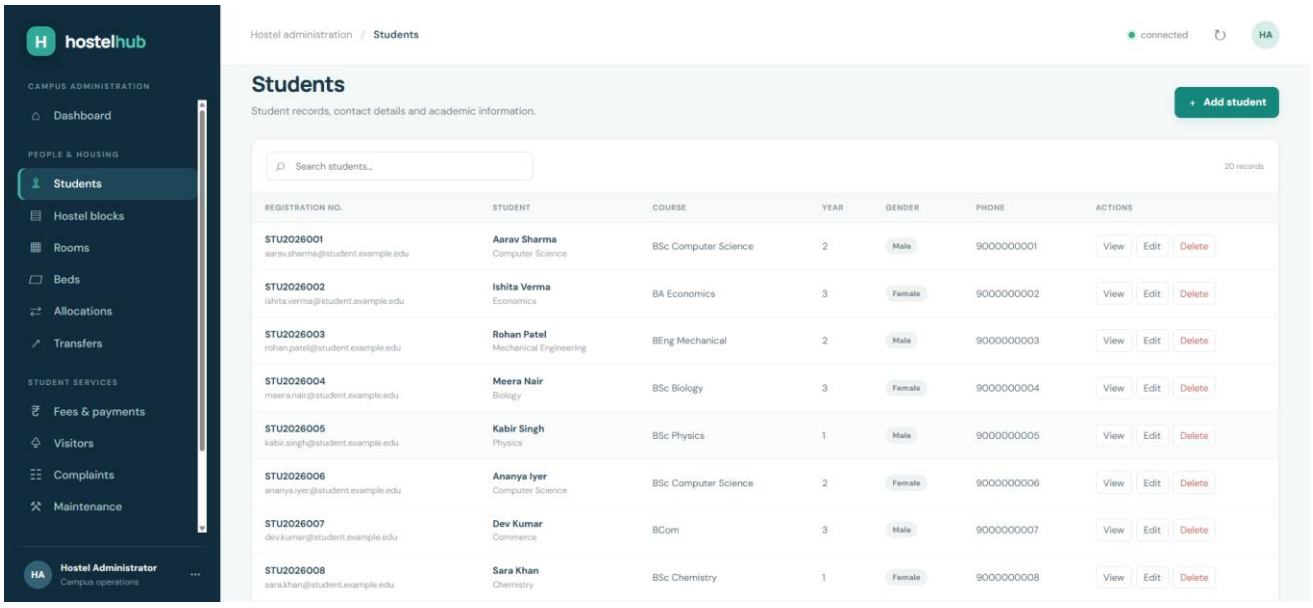


Figure C.2. C.2 Students Management

C.3 Hostel Blocks and Rooms

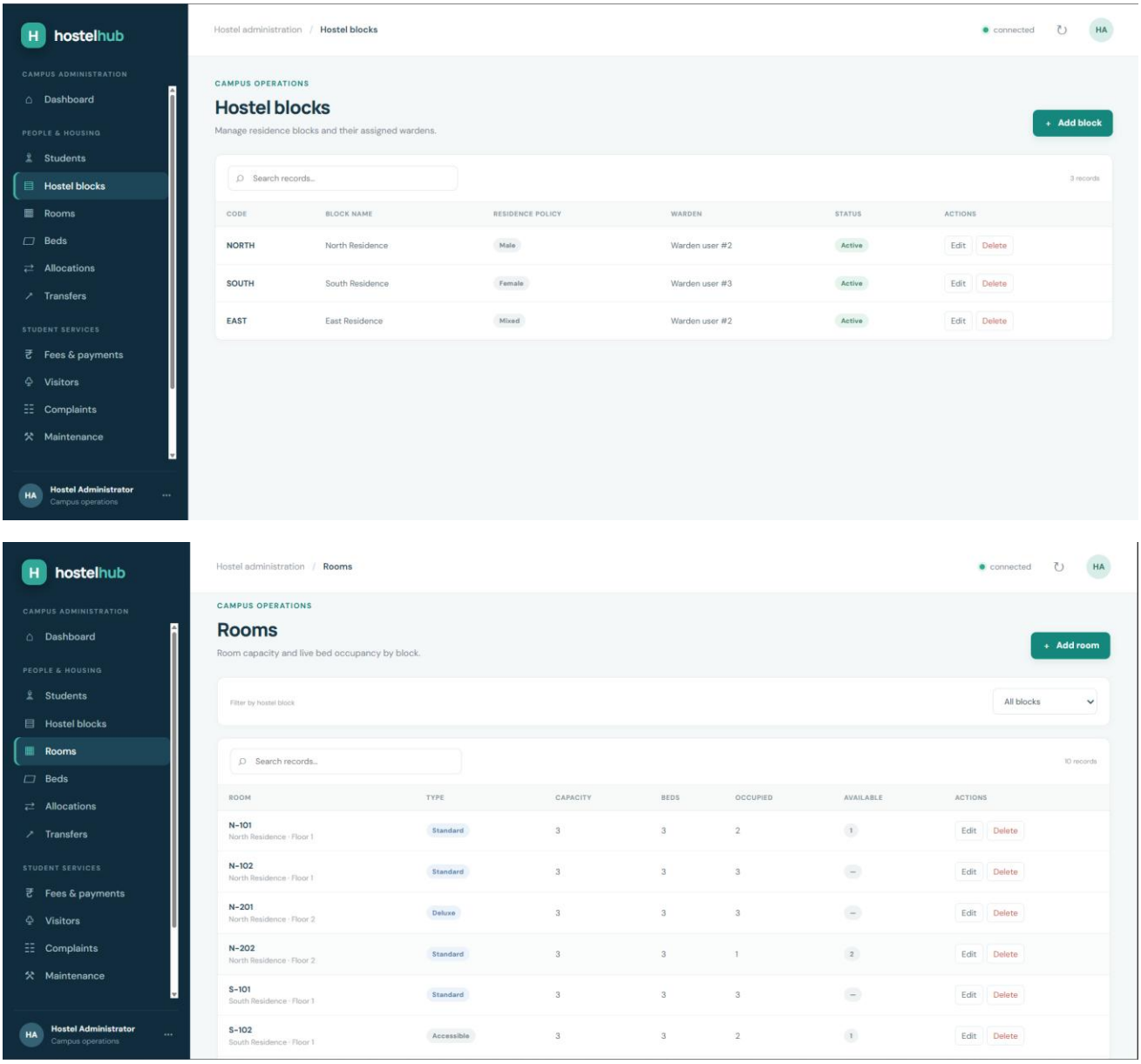


Figure C.3. C.3 Hostel Blocks and Rooms

C.4 Beds and Allocations

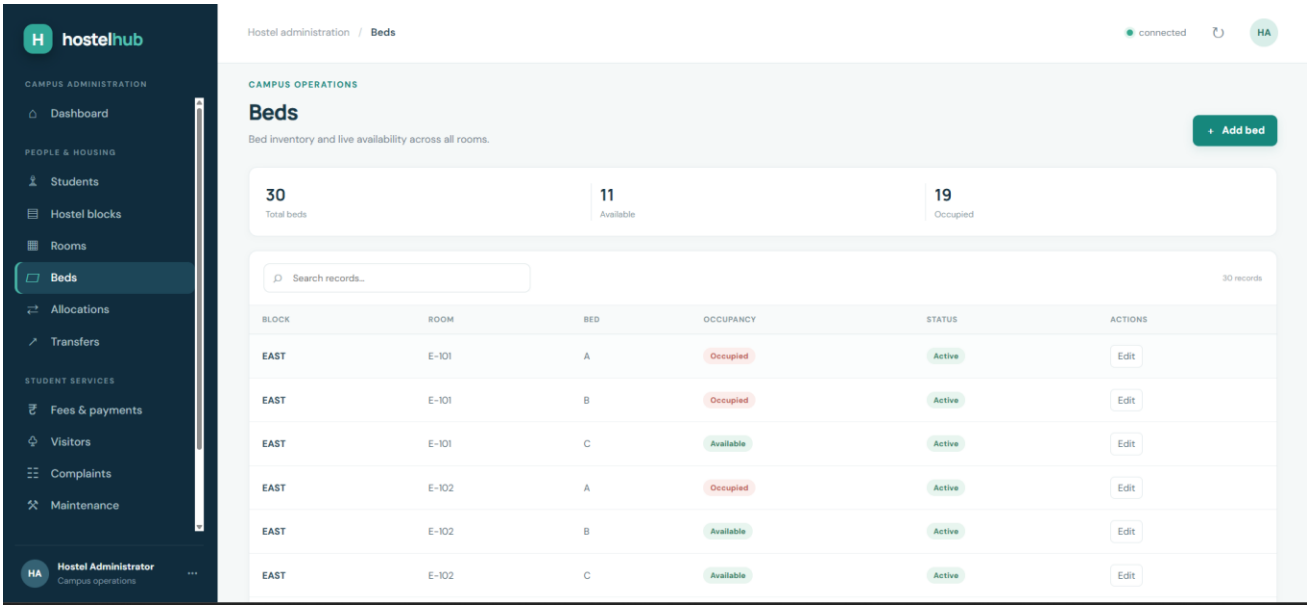


Figure C.4. C.4 Beds and Allocations

C.5 Fees and Payments

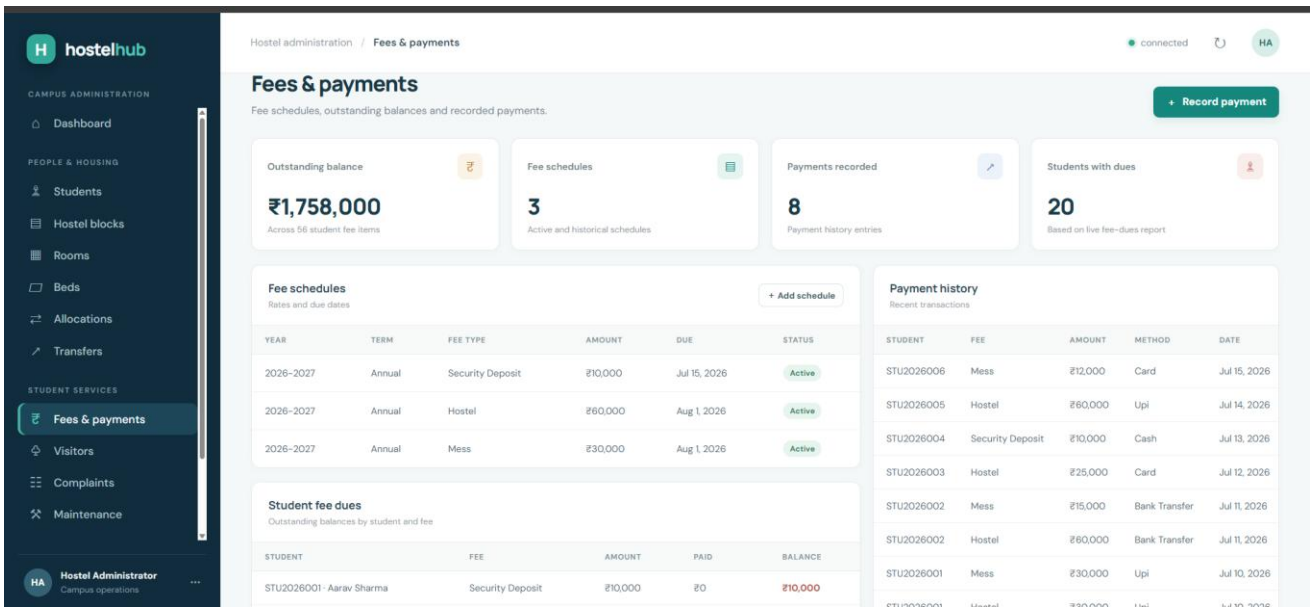


Figure C.5. C.5 Fees and Payments

C.6 Visitors

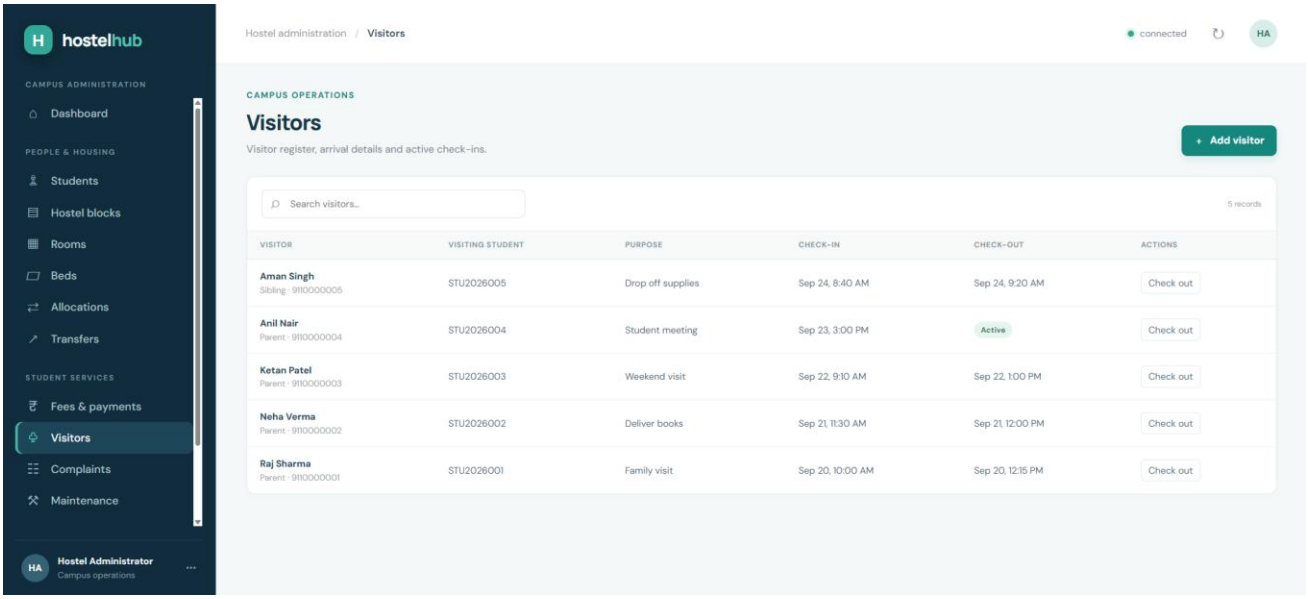


Figure C.6. C.6 Visitors

C.7 Complaints

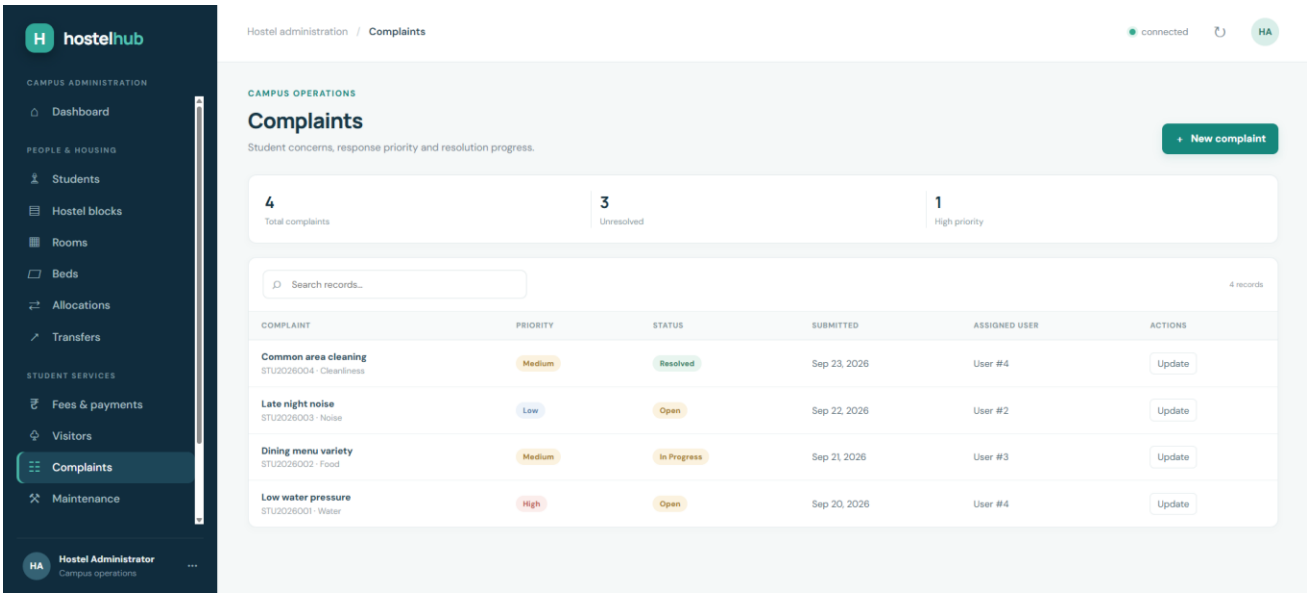


Figure C.7. C.7 Complaints

C.8 Maintenance

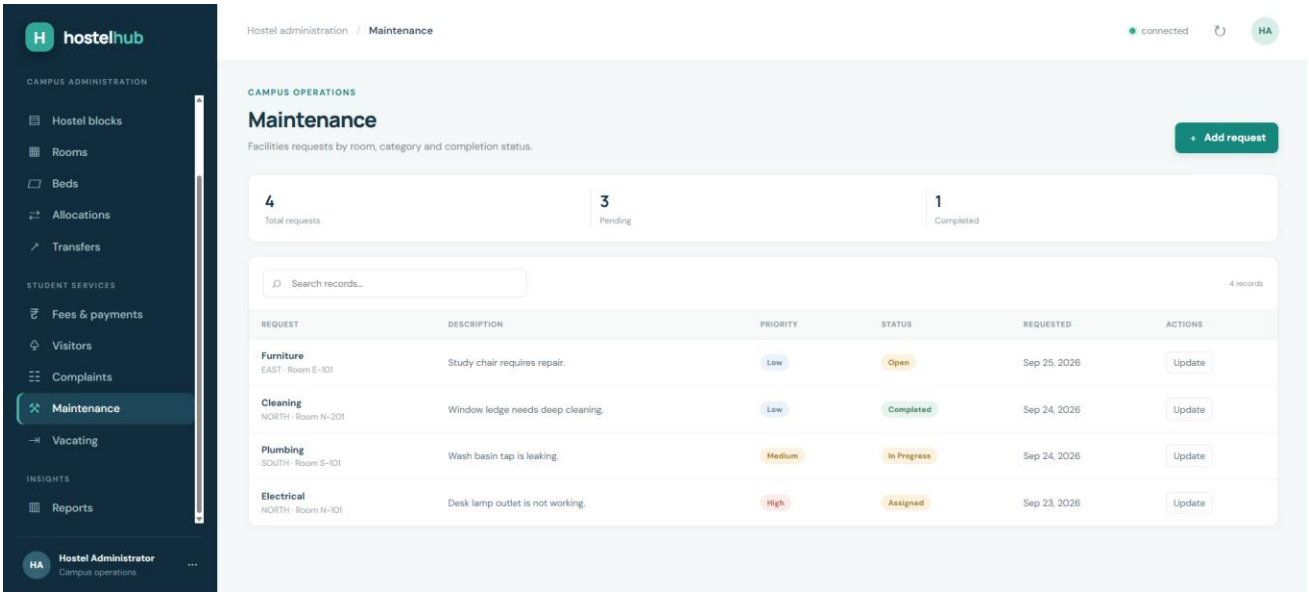


Figure C.8. C.8 Maintenance

C.9 Vacating

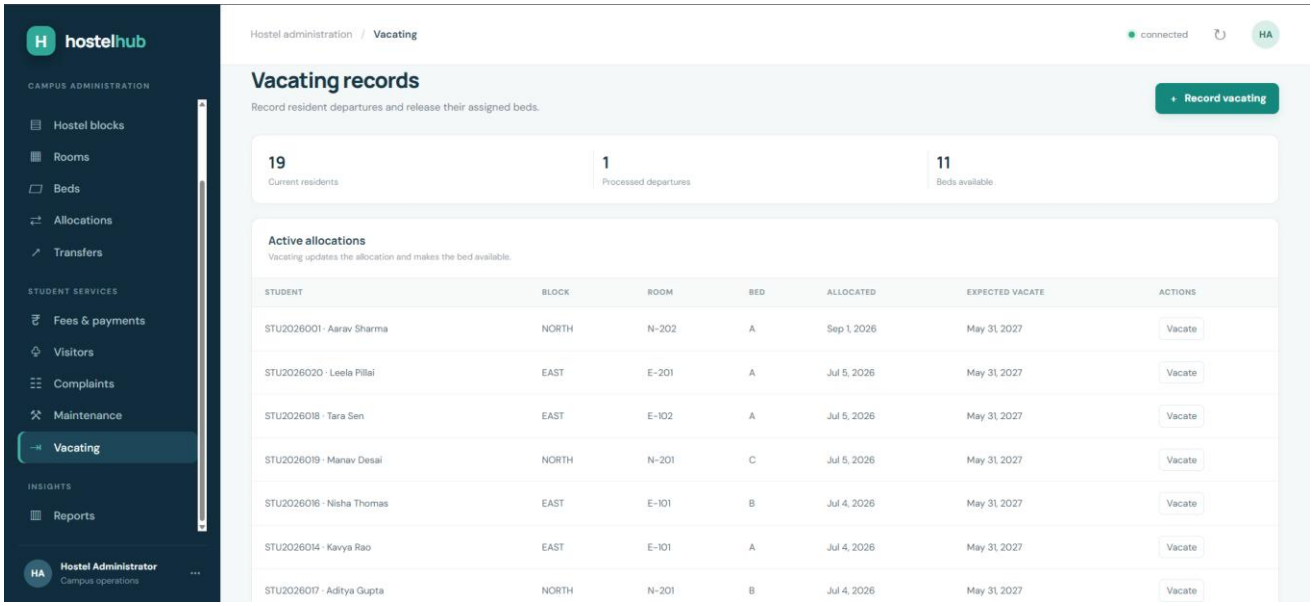


Figure C.9. C.9 Vacating

C.10 Reports

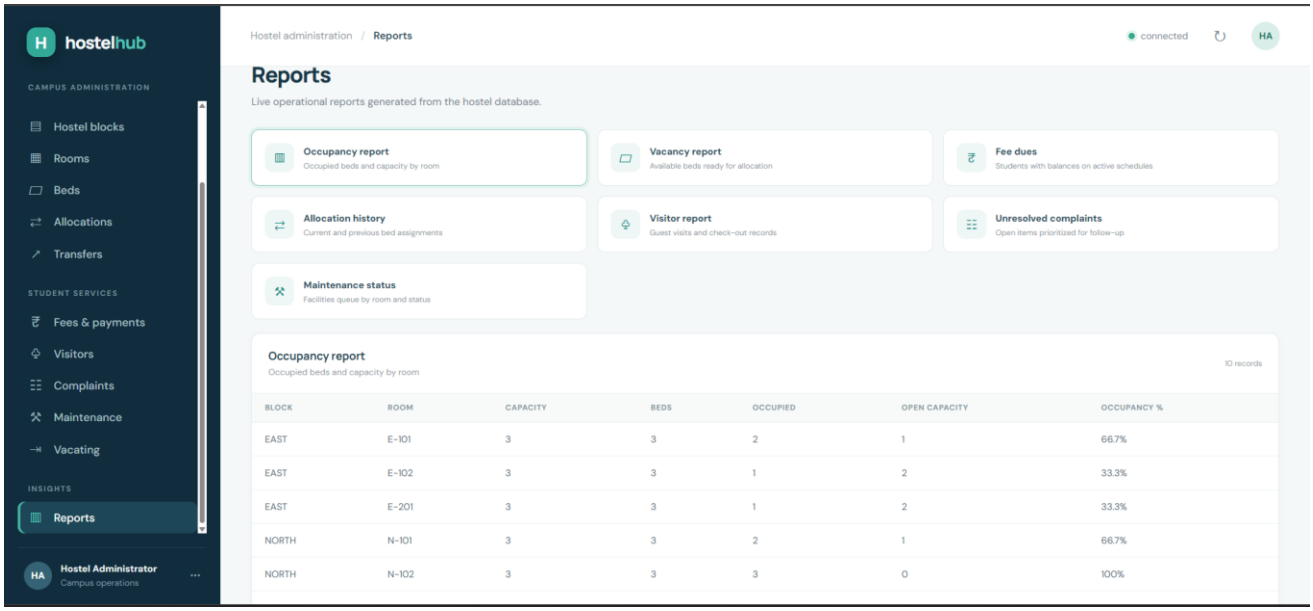
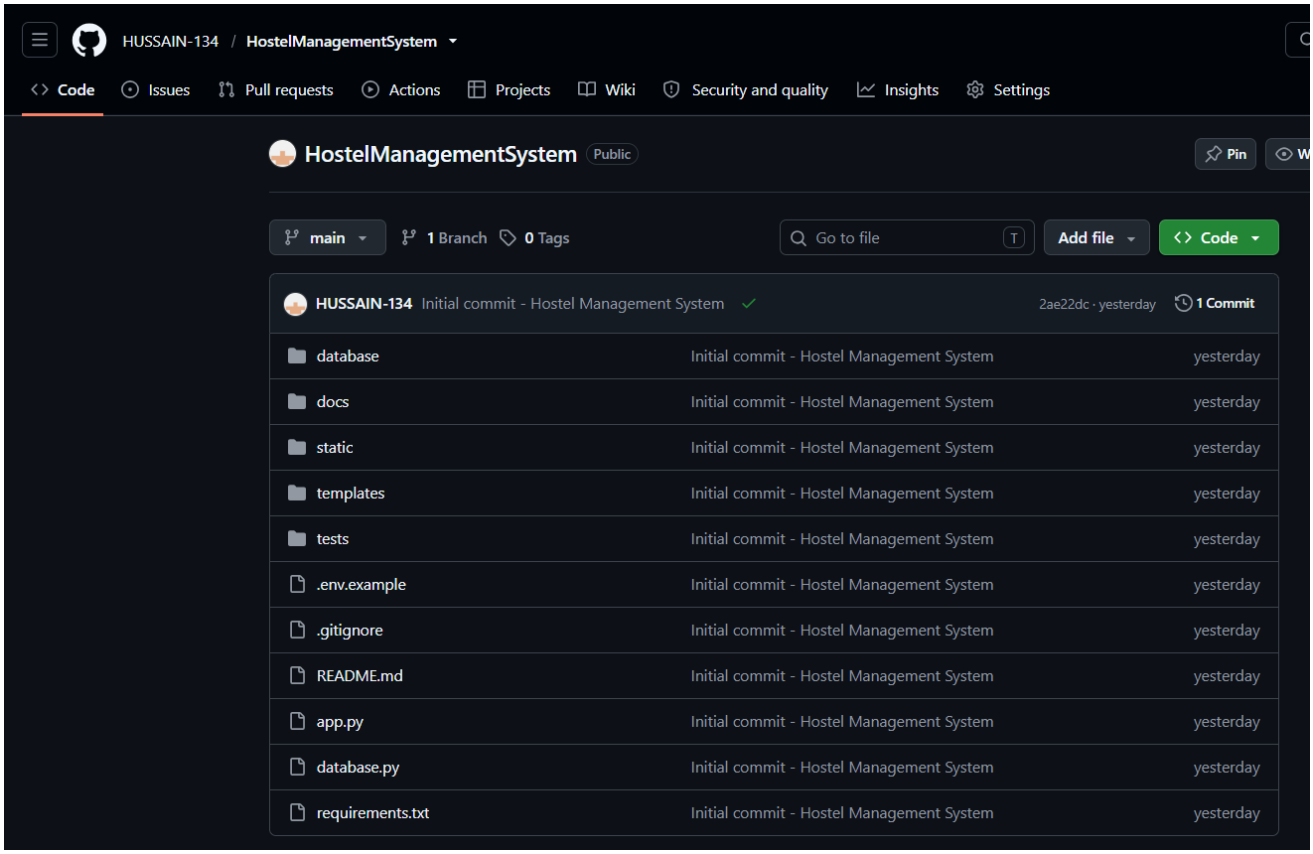


Figure C.10. C.10 Reports

D.5 GitHub Repository



D.5 GitHub Repository